



《AI大模型Ragent项目》——三态熔断器与故障转移

来自： 拿个offer-开源&项目实战



马丁

2026年04月09日 21:51

上一篇拆解了 ModelSelector 的选择算法，知道了它如何从一份 YAML 配置中选出一个有序的、可用的候选列表。但选好了之后呢？候选列表交给谁执行？调用第一个模型失败了怎么办？什么时候该认定一个模型挂了不再尝试？挂了的模型什么时候重新启用？这一篇回答这些问题。我们会深入两个核心组件：ModelHealthStore（三态熔断器）和 ModelRoutingExecutor（故障转移执行器）。前者管健康状态——谁能调、谁不能调；后者管执行逻辑——怎么调、失败了怎么切换。

三态熔断器：ModelHealthStore

1. 为什么需要熔断器

假设你的电商客服系统配了四个 Chat 模型：qwen3-max、glm-4.7、qwen-plus、qwen3-local。某天百炼平台出了故障，qwen3-max 连接超时。如果不做任何处理，每次用户提问都会先尝试调 qwen3-max，等 30 秒超时（或者直接失败），然后才切换到 glm-4.7。用户的每一个问题都要多等 30 秒——这显然不可接受。

熔断器的作用就是：连续失败达到阈值后，直接标记这个模型为不可用，后续请求跳过它，不再白等。等一段冷却时间过去，再试探性地放一个请求过去看看恢复了没有。恢复了就重新启用，没恢复就继续熔断。

这就是经典的断路器（Circuit Breaker）模式，ModelHealthStore 是它在项目中的实现。

2. 数据结构

ModelHealthStore 的核心是一个 ConcurrentHashMap，key 是模型 id，value 是这个模型的健康状态：

```
@Component
@RequiredArgsConstructor
public class ModelHealthStore {

    private final AIModelProperties properties;

    private final Map<String, ModelHealth> healthById = new ConcurrentHashMap<>();
    // ...
}
```

每个模型的健康状态由 ModelHealth 内部类表示：

```
private static class ModelHealth {
    private int consecutiveFailures; // 连续失败次数
    private long openUntil; // 熔断截止时间戳（毫秒）
    private boolean halfOpenInFlight; // 是否有探测请求在飞行中
    private State state; // 当前状态

    private ModelHealth() {
        this.consecutiveFailures = 0;
        this.openUntil = 0L;
        this.halfOpenInFlight = false;
        this.state = State.CLOSED;
    }
}
```

状态枚举只有三个值：

```
private enum State {
    CLOSED,
    OPEN,
    HALF_OPEN
}
```

新创建的 ModelHealth 默认是 CLOSED 状态，所有字段归零：

字段	初始值	含义
state	CLOSED	健康，正常放行
consecutiveFailures	0	连续失败次数，CLOSED 状态下每次失败 +1
openUntil	0L	熔断截止时间戳，OPEN 状态下有值
halfOpenInFlight	false	是否有探测请求正在执行，HALF_OPEN 状态下使用

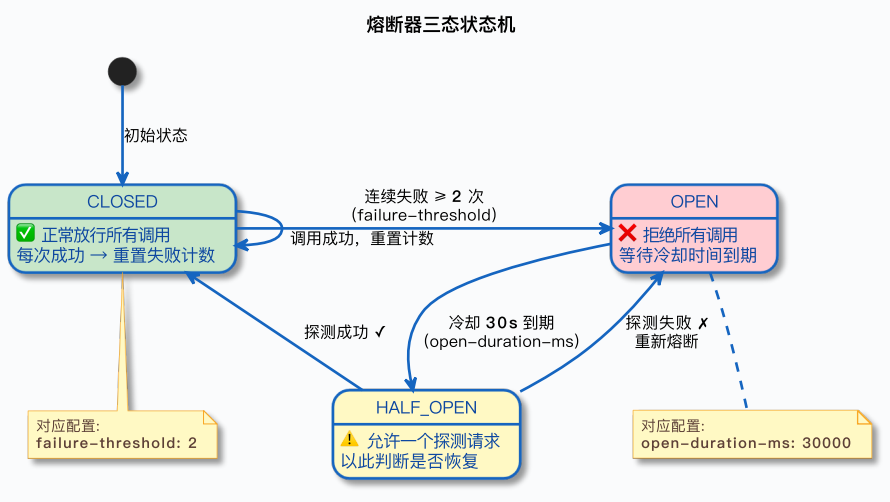
熔断行为由两个配置参数控制，来自 AIModelProperties.Selection：

- failureThreshold：连续失败多少次触发熔断，默认 2
- openDurationMs：熔断持续时间（毫秒），默认 30000（30 秒）

也就是说，默认配置下，一个模型连续失败 2 次就熔断，30 秒后才允许探测恢复。

3. 状态转换图

三态之间的转换关系用一张状态图表示：



四条转换边，每条都有明确的触发方法和条件。接下来逐个拆解这些方法的实现。

4. allowCall——放行检查

allowCall 是 ModelRoutingExecutor 在实际调用模型之前调用的检查方法。它不只是读状态——它还会在条件满足时修改状态（OPEN → HALF_OPEN 的转换就发生在这里）。

```
@SuppressWarnings("BooleanMethodIsAlwaysInverted")
public boolean allowCall(String id) {
    if (id == null) {
        return false;
    }
    long now = System.currentTimeMillis();
    AtomicBoolean allowed = new AtomicBoolean(false);
    healthById.compute(id, (k, v) -> {
        if (v == null) {
            v = new ModelHealth();
        }
        if (v.state == State.OPEN) {
            if (v.openUntil > now) {
                return v;
            }
            v.state = State.HALF_OPEN;
            v.halfOpenInFlight = true;
            allowed.set(true);
            return v;
        }
    });
    return allowed.get();
}
```

```
        if (v.state == State.HALF_OPEN) {
            if (v.halfOpenInFlight) {
                return v;
            }
            v.halfOpenInFlight = true;
            allowed.set(true);
            return v;
        }
        allowed.set(true);
        return v;
    });
    return allowed.get();
}
```

整个方法包在一个 ConcurrentHashMap.compute() 调用里，对同一个 key 的操作是原子的。lambda 内部的逻辑分五种情况：

当前状态	条件	操作	返回
无记录 (v == null)	—	创建初始 ModelHealth (CLOSED)	true (允许)
OPEN	openUntil > now (冷却未到)	不修改	false (拒绝)
OPEN	openUntil <= now (冷却已到)	转 HALF_OPEN, 设 halfOpenInFlight = true	true (允许探测)
HALF_OPEN	halfOpenInFlight = true (已有探测在飞)	不修改	false (拒绝)
HALF_OPEN	halfOpenInFlight = false (无探测在飞)	设 halfOpenInFlight = true	true (允许探测)
CLOSED	—	不修改	true (允许)

注意 AtomicBoolean allowed 的用法。compute 方法的返回值是 ModelHealth (Map 的 value 类型)，不是 boolean。lambda 内部没办法直接返回一个 boolean 给外层。所以用 AtomicBoolean 做中转——lambda 内部设置，lambda 外部读值。

为什么用 AtomicBoolean 而不是普通的 boolean[]? 因为 lambda 捕获的局部变量必须是 effectively final。AtomicBoolean 本身是 final 的引用，但它的值可以通过 set 修改，满足这个约束。用 boolean[] 也行，但 AtomicBoolean 语义更清晰。

4.1 为什么用 ConcurrentHashMap.compute()

ModelHealthStore 的所有状态修改方法 (allowCall、markSuccess、markFailure) 都用 ConcurrentHashMap.compute() 实现。为什么选这个方案？

对比三种可能的实现方式：

方案一：synchronized(this) 全局锁

```
public synchronized boolean allowCall(String id) {
    ModelHealth health = healthById.computeIfAbsent(id, k -> new ModelHealth());
    // 检查 + 修改状态
}
```

问题：不同模型的状态操作也互斥。线程 A 在操作 qwen3-max 的状态时，线程 B 对 glm-4.7 的操作也被阻塞了——它们之间完全没有竞争关系，不应该互相等待。并发度太低。

方案二：ConcurrentHashMap.get() + synchronized(health) 细粒度锁

```
public boolean allowCall(String id) {
    ModelHealth health = healthById.computeIfAbsent(id, k -> new ModelHealth());
    synchronized (health) {
        // 检查 + 修改状态
    }
}
```

看起来不错——锁在单个 `ModelHealth` 对象上，不同模型之间不互斥。但有一个隐含问题：`computeIfAbsent` 返回的对象和后续 `compute` 可能替换的对象不是同一个，如果其他代码路径替换了 `Map` 中的 `value`，你锁住的是旧对象。虽然当前实现不会替换 `value`（`compute` 总是返回同一个对象），但这种模式本身是脆弱的。

方案三： `ConcurrentHashMap.compute()` 原子操作

```
healthById.compute(id, (k, v) -> {
    // 在这里检查 + 修改状态，对同一个 key 不会并发执行
    return v;
});
```

`compute` 对同一个 `key` 的 `remappingFunction` 不会并发执行——这是 `ConcurrentHashMap` 的规范保证。不同 `key` 之间完全并发，不互相阻塞。代码最简洁，不需要显式 `lock/unlock`，不需要 `try-finally`。代价是 `lambda` 内不能抛受检异常、不能直接返回 `boolean`，需要用 `AtomicBoolean` 做中转。但相比其他方案，这点不便完全可以接受。用一个并发场景来体会 `compute` 的原子性保证：假设 `qwen3-max` 处于 `OPEN` 状态，冷却时间刚到。线程 A（处理用户问题 1）和线程 B（处理用户问题 2）几乎同时调用 `allowCall("qwen3-max")`。如果不是原子的：

- 1. 线程 A 读到 `OPEN` + 冷却已到 → 准备转 `HALF_OPEN`
- 2. 线程 B 也读到 `OPEN` + 冷却已到 → 也准备转 `HALF_OPEN`
- 3. 两个线程都把状态改成 `HALF_OPEN`，都设了 `halfOpenInFlight = true`，都返回了 `true`
- 4. 两个探测请求同时涌入刚出故障的供应商——但 `HALF_OPEN` 的设计意图是只放一个

用 `compute` 就不会有这个问题：线程 A 先进入 `lambda`，把状态改成了 `HALF_OPEN` 并设置 `halfOpenInFlight = true`；线程 B 进入 `lambda` 时看到的已经是 `HALF_OPEN + halfOpenInFlight = true`，直接被拒绝。

4.2 HALF_OPEN 为什么只允许一个探测请求

`HALF_OPEN` 状态的核心约束是：同一时刻只允许一个探测请求通过。这是 `halfOpenInFlight` 标志的作用。为什么不能多放几个？因为探测请求是拿真实用户的请求去试的——不是后台发一个心跳包，而是某个用户实实在在地在等响应。如果供应商还没恢复，每多放一个探测请求，就多一个用户要承受超时等待的代价。虽然 `ModelRoutingExecutor` 有 `fallback` 逻辑，探测失败后会尝试下一个模型，但这个用户的响应延迟已经多了一轮超时时间（比如 30 秒）。放 10 个探测请求进去，就是 10 个用户白等。只放一个探测请求是最保守的策略：用最小的代价验证供应商是否恢复。如果成功，`markSuccess` 重置为 `CLOSED`，后续所有请求正常通过；如果失败，`markFailure` 重新熔断，再等一轮冷却。牺牲的只是一个用户的一轮超时，而不是一批。

5. markSuccess——成功恢复

`markSuccess` 的逻辑非常直接：不管当前是什么状态，全部重置为初始值。

```
public void markSuccess(String id) {
    if (id == null) {
        return;
    }
    healthById.compute(id, (k, v) -> {
        if (v == null) {
            return new ModelHealth();
        }
        v.state = State.CLOSED;
        v.consecutiveFailures = 0;
        v.openUntil = 0L;
        v.halfOpenInFlight = false;
        return v;
    });
}
```

它最重要的使用场景是 `HALF_OPEN` 状态下的探测成功。探测请求调用模型成功了，`executeWithFallback` 内部会调 `markSuccess`，把状态从 `HALF_OPEN` 重置为 `CLOSED`，模型重新回到正常服务。在 `CLOSED` 状态下调 `markSuccess` 也没问题——它会把 `consecutiveFailures` 清零。这意味着每一次成功调用都会重置失败计数器，避免偶发性的单次失败被累计。比如 `qwen3-max` 第一次调用失败了（`consecutiveFailures = 1`），第二次调用成功了（`consecutiveFailures` 被重置为 0），那么第三次再失败时，计数器是从 0 开始的，不会和第一次的失败叠加。

6. markFailure——失败处理

markFailure 是最复杂的方法，有两条分支逻辑：

```
public void markFailure(String id) {
    if (id == null) {
        return;
    }
    long now = System.currentTimeMillis();
    healthById.compute(id, (k, v) -> {
        if (v == null) {
            v = new ModelHealth();
        }
        if (v.state == State.HALF_OPEN) {
            v.state = State.OPEN;
            v.openUntil = now + properties.getSelection().getOpenDurationMs();
            v.consecutiveFailures = 0;
            v.halfOpenInFlight = false;
            return v;
        }
        v.consecutiveFailures++;
        if (v.consecutiveFailures >= properties.getSelection().getFailureThreshold()) {
            v.state = State.OPEN;
            v.openUntil = now + properties.getSelection().getOpenDurationMs();
            v.consecutiveFailures = 0;
        }
        return v;
    });
}
```

分支一：HALF_OPEN 状态下失败。 探测请求失败了，说明供应商还没恢复。一票否决——直接重新 OPEN，设置新的 openUntil（当前时间 + 冷却时间），重新开始倒计时。同时重置 consecutiveFailures = 0 和 halfOpenInFlight = false。

为什么重置 consecutiveFailures？因为 HALF_OPEN → OPEN 的转换不依赖累计次数，它是一票否决的。consecutiveFailures 只在 CLOSED 状态下有意义——用来判断是否达到阈值触发熔断。HALF_OPEN 失败后重新进入 OPEN，等下次再回到 CLOSED 时，计数器应该从 0 开始。

分支二：CLOSED 状态下失败（默认分支）。 consecutiveFailures++，然后检查是否达到阈值。如果 consecutiveFailures >= failureThreshold（默认 2），触发熔断：state = OPEN，设置 openUntil，重置 consecutiveFailures = 0。如果没达到阈值，只是计数器 +1，状态不变。

注意：OPEN 状态下理论上不会有请求进来（allowCall 会拒绝），所以 markFailure 不需要显式处理 OPEN 状态。代码中 OPEN 状态会走默认分支（consecutiveFailures++），但这不影响行为——因为 OPEN 状态下 consecutiveFailures 已经被重置为 0，而且不会有连续的 markFailure 调用。

7. 时间线走查：从正常到熔断到恢复

用一个完整的时间线走查 qwen3-max 模型的状态变化，配置为默认值（failureThreshold = 2, openDurationMs = 30000）：

时刻	事件	触发方法	state	consecutiveFailures	openUntil	halfOpenInFlight
T=0s	初始状态，正常服务	—	CLOSED	0	0	false
T=1s	第一次调用失败	markFailure	CLOSED	1	0	false
T=2s	第二次调用失败，达到阈值	markFailure	OPEN	0	T+30s	false
T=3s~T=31s	所有请求被跳过	allowCall → false	OPEN	0	T=32s	false
T=32s	冷却时间到，放行一个探测	allowCall	HALF_OPEN	0	T=32s	true

到这里，命运分两条路：

场景 A：探测成功

时刻	事件	触发方法	state	consecutiveFailures	openUntil	halfOpenInFlight
----	----	------	-------	---------------------	-----------	------------------

T=33s	探测请求成功	markSuccess	CLOSED	0	0	false
T=34s+	恢复正常服务	allowCall → true	CLOSED	0	0	false

场景 B：探测失败

时刻	事件	触发方法	state	consecutiveFailures	openUntil	halfOpenInFlight
T=33s	探测请求失败	markFailure	OPEN	0	T=63s	false
T=34s~T=62s	再次进入冷却	allowCall → false	OPEN	0	T=63s	false
T=63s	再次尝试探测	allowCall	HALF_OPEN	0	T=63s	true

场景 B 会循环下去，直到某次探测成功才回到 CLOSED。每次失败后重新开始 30 秒冷却倒计时。

8. 方法一览

用一张表格总结 ModelHealthStore 的四个公开方法：

方法	调用时机	是否修改状态	核心行为
isUnavailable(id)	ModelSelector 构建候选列表时	否（只读）	OPEN 且冷却未到 → 不可用；HALF_OPEN 且有探测在飞 → 不可用
allowCall(id)	ModelRoutingExecutor 实际调用前	是	OPEN 冷却到期 → 转 HALF_OPEN；HALF_OPEN 无探测 → 允许探测
markSuccess(id)	模型调用成功后	是	重置为初始 CLOSED 状态
markFailure(id)	模型调用失败后	是	HALF_OPEN → 重新 OPEN；CLOSED → 累计失败，达阈值则 OPEN

前两个是检查方法（上一篇讲了它们的两层机制），后两个是标记方法。executeWithFallback 在调用模型成功后调 markSuccess，失败后调 markFailure——这就是熔断器和执行器之间的协作方式。

同步调用的故障转移：ModelRoutingExecutor

1. executeWithFallback 完整代码

ModelRoutingExecutor 是一个很轻的组件，只注入了 ModelHealthStore，没有注入 ModelSelector——候选列表由调用方传入。它是一个纯粹的执行器，不做选择，只做执行和故障转移。

```
@Slf4j
@Component
@RequiredArgsConstructor
public class ModelRoutingExecutor {

    private final ModelHealthStore healthStore;

    public <C, T> T executeWithFallback(
        ModelCapability capability,
        List<ModelTarget> targets,
        Function<ModelTarget, C> clientResolver,
        ModelCaller<C, T> caller) {
        String label = capability.getDisplayName();
        if (targets == null || targets.isEmpty()) {
            throw new RemoteException("No " + label + " model candidates available");
        }

        Throwable last = null;
        for (ModelTarget target : targets) {
```

```
C client = clientResolver.apply(target);
if (client == null) {
    log.warn("{} provider client missing: provider={}, modelId={}",
        label, target.candidate().getProvider(), target.id());
    continue;
}
if (!healthStore.allowCall(target.id())) {
    continue;
}

try {
    T response = caller.call(client, target);
    healthStore.markSuccess(target.id());
    return response;
} catch (Exception e) {
    last = e;
    healthStore.markFailure(target.id());
    log.warn("{} model failed, fallback to next. modelId={}, provider={}",
        label, target.id(), target.candidate().getProvider(), e);
}

throw new RemoteException(
    "All " + label + " model candidates failed: "
        + (last == null ? "unknown" : last.getMessage()),
    last,
    BaseErrorCode.REMOTE_ERROR
);
}
```

逻辑用一句话概括：遍历候选列表，逐个尝试，成功就返回，失败就切换到下一个，全部失败就抛异常。
遍历过程中每个候选经过三关：

- 1. **客户端查找**：clientResolver.apply(target) 根据 ModelTarget 查找对应的客户端实例。如果找不到（返回 null），说明这个供应商的客户端没有注册，打印警告日志然后跳过。
- 2. **熔断检查**：healthStore.allowCall(target.id()) 做调用阶段的最终判断。如果模型被熔断（返回 false），直接跳过，连调用都不发起。
- 3. **实际调用**：caller.call(client, target) 执行真正的模型调用。成功了 markSuccess 后返回；失败了 markFailure 后继续尝试下一个候选。

2. 泛型设计：ModelCaller<C, T>

executeWithFallback 是一个泛型方法，C 是客户端类型，T 是返回类型。实际的调用逻辑由 ModelCaller<C, T> 函数式接口提供：

```
@FunctionalInterface
public interface ModelCaller<C, T> {
    T call(C client, ModelTarget target) throws Exception;
}
```

为什么要这样设计？因为 executeWithFallback 要服务三种能力——Chat、Embedding、Rerank——它们的客户端类型和返回类型都不一样：

能力	C（客户端类型）	T（返回类型）	调用代码
Chat	ChatClient	String	client.chat(request, target)
Embedding	EmbeddingClient	List<Float>	client.embed(text, target)
Rerank	RerankClient	List<RetrievedChunk>	client.rerank(query, candidates, topN, target)

如果不用泛型，executeWithFallback 内部就得 switch 能力类型，每新增一种能力就要改执行器代码——违反开闭原则。用泛型 + 函数式接口，新增能力只需要传入对应的 clientResolver 和 caller，执行器本身一行代码都不用改。

ModelCaller 的 call 方法声明了 throws Exception。这是有意为之的：模型调用可能抛出各种异常（网络超时、HTTP 错误、JSON 解析失败），executeWithFallback 在 catch 块里统一处理，不需要调用方自己 try-catch。

3. 一行代码串起整条链路

看看 RoutingLLMService.chat() 如何用一行代码把 ModelSelector 选择、客户端查找、实际调用、故障转移全部串起来：

```
@Override
@RagTraceNode(name = "llm-chat-routing", type = "LLM_ROUTING")
public String chat(ChatRequest request) {
    return executor.executeWithFallback(
        ModelCapability.CHAT,
        selector.selectChatCandidates(Boolean.TRUE.equals(request.getThinking())),
        target -> clientsByProvider.get(target.candidate().getProvider()),
        (client, target) -> client.chat(request, target)
    );
}
```

四个参数各司其职：

ModelCapability.CHAT——告诉执行器这是 Chat 能力调用，日志里会显示 "Chat" 标签

selector.selectChatCandidates(...)——从 ModelSelector 拿到已排序的候选列表（上一篇讲的选择算法）

target -> clientsByProvider.get(target.candidate().getProvider())——根据候选的供应商名称查找对应的 ChatClient 实例

(client, target) -> client.chat(request, target)——实际的 Chat 调用逻辑

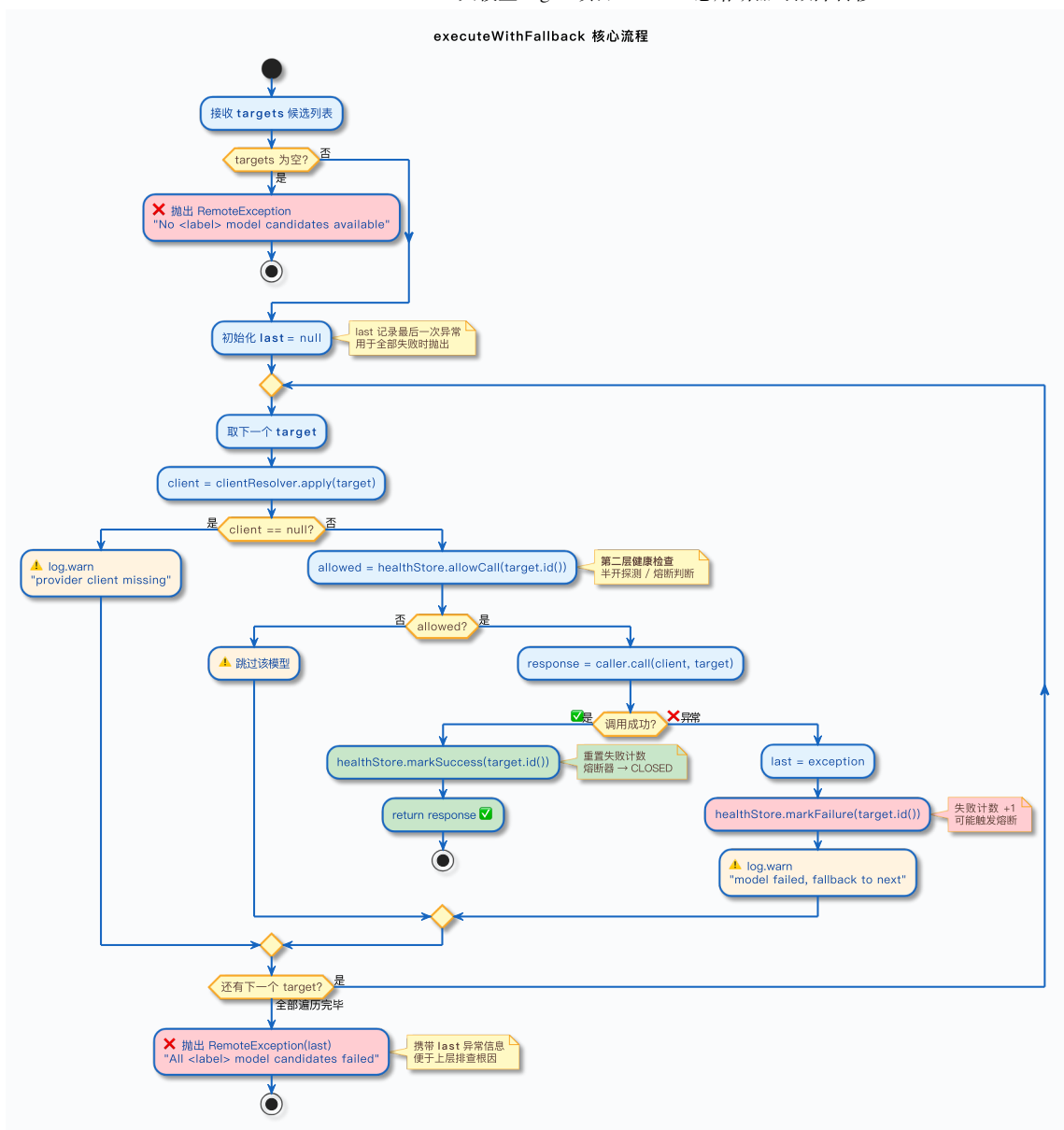
再看 RoutingEmbeddingService.embed()——同一个执行器，换了能力类型和调用逻辑：

```
@Override
public List<Float> embed(String text) {
    return executor.executeWithFallback(
        ModelCapability.EMBEDDING,
        selector.selectEmbeddingCandidates(),
        this::resolveClient,
        (client, target) -> client.embed(text, target)
    );
}
```

模式完全一样：选候选 → 找客户端 → 调接口 → 故障转移。一套 executeWithFallback 机制，三种能力共用。

4. 执行流程图

用一张活动图展示 executeWithFallback 的完整流程：



5. 完整场景走查：故障转移 + 熔断联动

把熔断器和执行器串起来，走一个完整的场景。还是电商客服知识库的例子，候选列表是：qwen3-max → glm-4.7 → qwen-plus → qwen3-local。

第一次请求：qwen3-max 超时，切换到 glm-4.7

用户问：AirPods Pro 2 的保修期是多久？

1. RoutingLLMService.chat() 调用 executor.executeWithFallback()
2. 遍历候选列表，第一个是 qwen3-max
3. clientResolver 找到百炼的 ChatClient → 非 null
4. healthStore.allowCall("qwen3-max") → 初始 CLOSED，返回 true
5. caller.call(client, target) → 百炼平台超时，抛出异常
6. healthStore.markFailure("qwen3-max") → consecutiveFailures = 1，仍然 CLOSED（未达到阈值 2）
7. 继续下一个候选 glm-4.7
8. healthStore.allowCall("glm-4.7") → CLOSED，返回 true
9. caller.call(client, target) → 成功返回答案
10. healthStore.markSuccess("glm-4.7") → 保持 CLOSED

用户拿到了 glm-4.7 的回答，延迟多了一次超时等待，但最终成功了。

第二次请求：qwen3-max 再次失败，触发熔断

用户问：能退货吗？

1. ModelSelector.selectChatCandidates() 选出候选列表——qwen3-max 的 isUnavailable 返回 false（还在 CLOSED），所以它仍然在列表中

2. `executeWithFallback` 遍历，第一个还是 `qwen3-max`
3. `allowCall("qwen3-max")` → `CLOSED`，返回 `true`
4. `caller.call(client, target)` → 百炼平台仍然超时
5. `markFailure("qwen3-max")` → `consecutiveFailures = 2 >= threshold`，**触发熔断** → `OPEN`，`openUntil = now + 30s`
6. 继续下一个候选 `glm-4.7`，成功返回

第三次请求：qwen3-max 被跳过

用户问：运费怎么算？

1. `ModelSelector.selectChatCandidates()` 选出候选列表——这次 `isUnavailable("qwen3-max")` 返回 `true`（`OPEN` 且冷却未到），`qwen3-max` 被预过滤掉，不出现在候选列表中
2. 候选列表变成了：`glm-4.7` → `qwen-plus` → `qwen3-local`
3. `executeWithFallback` 直接从 `glm-4.7` 开始，成功返回

从第三次请求开始，用户不再需要等 `qwen3-max` 的超时了。30 秒后 `qwen3-max` 进入 `HALF_OPEN`，允许一个探测请求通过。如果百炼平台恢复了，探测成功，`qwen3-max` 回到 `CLOSED`，重新作为默认模型服务。

整个过程对业务层完全透明——`RoutingLLMService.chat()` 总是返回一个成功的回答，调用方不需要知道背后发生了什么切换。

小结与下一步

回顾这一篇的核心要点：

`ModelHealthStore` 用 `ConcurrentHashMap.compute()` 实现了线程安全的三态熔断器，每个模型独立计状态，互不影响

`CLOSED` → `OPEN` → `HALF_OPEN` 三态转换：连续失败达到阈值触发熔断 → 冷却后允许一个探测请求 → 探测结果决定恢复或继续熔断

`HALF_OPEN` 只允许一个探测请求通过，避免故障恢复时的请求雪崩

`ModelRoutingExecutor.executeWithFallback()` 用泛型 + 函数式接口实现了通用的故障转移执行器，`Chat` / `Embedding` / `Rerank` 三种能力共用一套逻辑

两层熔断检查（选择阶段 `isUnavailable` + 调用阶段 `allowCall`）配合故障转移执行器，让同步调用具备完整的容错能力

不过，`executeWithFallback` 只能处理同步调用。流式调用的情况更复杂——`client.streamChat()` 返回的是一个 `StreamCancellationHandle`，真正的数据通过 `StreamCallback` 异步推送，调用返回的瞬间你不知道供应商能不能正常工作。这需要一套专门的首包探测机制来解决，后续文章会详细讲。

下一篇我们先进入 **Chat 同步调用与模板方法**——深入 `Chat` 子系统的实现：`ChatClient` / `LLMService` 的接口分层设计，`AbstractOpenAIStyleChatClient` 如何用模板方法封装 `OpenAI` 兼容协议的请求构建与响应解析，三个供应商客户端（百炼、硅基流动、Ollama）各自需要覆写的差异点，以及 `HTTP` 基础设施（`URL` 解析、异常体系、错误分类）。